Informe de Laboratorio 07

Tema: Autenticación, Sesiones y Seguridad en Express

Nota

Estudiantes:	Escuela	Asignatura
Josemiguel Pineda Sucasaca Fabiano Enriquez Molina Piero Arosquipa Laury Cesar Coricaza Lopez	Escuela Profesional de Ingeniería de Sistemas	Introduccion al Desarrollo web Semestre: II Código: 2501210

Laboratorio	Tema	Duración
07	Autenticación, Sesiones y Seguridad en Express	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	Del 12 Octubre 2026	Al 20 Octubre 2026

1. Ejercicios / Problemas Propuestos

1.1. Caso Práctico: Sistema de Autenticación y Rutas Privadas Académicas

Implementar un subsistema completo de seguridad en Express cumpliendo los siguientes ítems:

1.1.1. Gestión de Variables de Entorno:

- Crear un archivo .env que contenga PORT=3000 y SESSION_SECRET=clave_secreta_super_segura. Cargarlas en Express usando dotenv.

1.1.2. Registro e Inicio de Sesión:

- Endpoint POST /api/auth/register: Recibe email y password. Aplica hash asíncrono con bcryptjs (10 salt rounds) antes de guardar el usuario en un arreglo en memoria.
- Endpoint POST /api/auth/login: Valida las credenciales comparando los hashes mediante bcrypt.compare(). Si es exitoso, crea la propiedad req.session.userId = user.id.

1.1.3. Cierre de Sesión y Recurso Protegido:

- Middleware authMiddleware: Verifica si req.session.userId existe. Si no existe, interrumpe el flujo y retorna 401 Unauthorized.

- Endpoint Protegido GET /api/perfil: Protegido por authMiddleware. Retorna los datos del usuario autenticado.
- Endpoint POST /api/auth/logout: Destruye la sesión del servidor (req.session.destroy()) y limpia la cookie de sesión.

2. Equipos, materiales y temas utilizados

- Postman versión 12.31.7
- Version de Android - 16
- Sistema Operativo Microsoft Windows 11 version 25H2.
- Version del Visual Code: 1.138.0 - 7debcd0e2acdea1c52de81bf9ee1620444407dda - x64
- Bloc de notas de Windows 11.2607.14.0.
- Identidad del navegador: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 OPR/135.0.0.0.
- Git 2.55.0.
- Uso de Git-hub.
- Cuenta en GitHub con el correo institucional.
- Programación para el Desarrollo Web .
- Terminal /CLI
- Usando comandos npm

3. URL de Repositorio Github

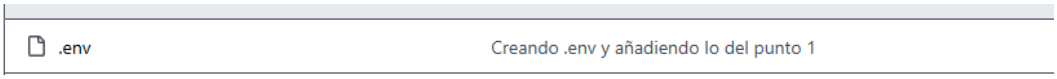
- URL del Repositorio GitHub para clonar o recuperar.
- `https://github.com/jpinedasu/nsk.git`

4. Actividades a realizar según guía de laboratorio

4.1. Gestión de Variables de Entorno:

- **1.** Crear un archivo .env que contenga PORT=3000 y SESSION_SECRET=clave_secreta_super_segura. Cargarlas en Express usando dotenv.

Figura 1: visualizacion del archivo .env en el github



- A continuación se mostrara cual es el codigo dentro del archivo .env:

```
PORT=3000
SESSION_SECRET=clave_secreta_super_segura
```

- Se usa **PORT=3000** para indicar el puerto donde se correra el codigo, en la barra de navegacion aparecera de esta manera: **http://localhost:3000**, y luego en el archivo node.js se muestra el siguiente codigo para definirlo:

```
const PORT = process.env.PORT || 3000;
```

- Se usa **SESION_SECRET** ya que esta es una **variable de entorno** la cual garantiza que el cliente no pueda alterar la identidad o los datos guardados en la sesión.
- Esta es parte de dotenv y express (**Se tiene que descargar**), pero para que funcione se tiene que configurar en el server.js del archivo, para ello a continuacion se mostrara como se muestra en el codigo.

```
// dotenv al inicio para usar PORT y SESSION SECRET dsde .env

require('dotenv').config();
const express = require('express');
const session = require('express-session');
const bcrypt = require('bcryptjs');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

// Configurar la sesion con la clave secreta super segura y opciones de seguridad
app.use(session({
    secret: process.env.SESSION_SECRET || 'clave_secreta_super_segura',
    resave: false,
    saveUninitialized: false,
    cookie: {
        httpOnly: true, // Protege la cookie contra ataques XSS
        secure: false,
        maxAge: 1000 * 60 * 60 // Se mantiene 1 hora la sesin
    }
}));
```

- Se puede observar en la parte final del codigo la configuracion de la cookie, esta es necesaria para cifrarla, y alguien no pueda acceder a ella de una manera facil, para ello se coloco **httpOnly true** para que la cookie este protegida contra ataques **XSS** esto sirve para que no se inyecten scripts malisiosos, a la vez que no haya una parte vulnerable en el codigo, luego se usa **maxAge** donde se define que la sesion dure por 1 hora, para proteger el sitio web.

4.2. Registro e Inicio de Sesión:

- **1.** Endpoint POST /api/auth/register: Recibe email y password. Aplica hash asíncrono con bcryptjs (10 salt rounds) antes de guardar el usuario en un arreglo en memoria.
- DEMOSTRACION DEL CODIGO CUANDO SE RECIBE EL USUARIO Y CONTRASEÑA, FILTROS BASICOS.

```
// Endpoint POST /api/auth/register
// Recibe email y contrasena
app.post('/api/auth/register', async (req, res) => {
```

```
    try {
        const { email, password } = req.body;

        if (!email || !password) {
            return res.status(400).json({ status: 'fail', message: 'Email y contrasena
                requeridos' });
        }

        const existingUser = users.find(u => u.email === email);
        if (existingUser) {
            return res.status(400).json({ status: 'fail', message: 'El usuario ya esta
                registrado' });
        }
```

- **Explicacion del codigo:** Como se definio express como app, se define el **POST** a traves de ella, y se coloca la ruta que se pidio para el registro, luego se definen los parametros que tiene esta ruta, luego se crean las constantes email y password (contraseña), donde se colocara la informacion, antes de subirlo se verifican 2 puntos:
- **Punto 1:** Se coloca un filtro de seguridad por si el usuario no coloca ambas partes en los campos requeridos, y si pasa eso se devuelve **codigo 400**, indicando que complete los campos.
- **Punto 2:** Se tiene que evitar la **duplicidad** (si hay 2 usuarios con el mismo email), para ello se crea **const existingUser**, el cual compara el email que se esta registrando con el que ya se tenia en la base de datos, si esta encuentra que si existe, se lanza automaticamente **codigo 400** y un mensage que indica: El usuario ya esta registrado.
- Luego de estos filtros basicos se tiene que cifrar la contraseña de forma asincronica. DEMOSTRACION DEL CODIGO:

```
        // Cifrado asncrono de la contrasea
        const hashedPassword = await bcrypt.hash(password, 10);

        const newUser = {
            id: Date.now().toString(),
            email,
            password: hashedPassword
        };

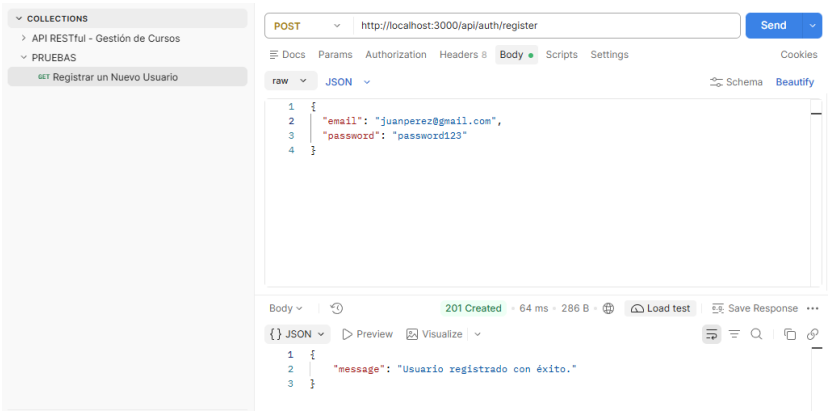
        users.push(newUser);

        return res.status(201).json({
            status: 'success',
            message: 'Usuario registrado exitosamente',
            user: { id: newUser.id, email: newUser.email }
        });
    } catch (error) {
        return res.status(500).json({ status: 'error', message: 'Error interno en el
            servidor' });
    }
});
```

- **Explicacion del codigo:** se crea **const hashedPassword** done se usa **bcrypt**, se colocan los parametros, en una parte esta la contraseña y en la otra **el numero de salt round (son 10)**, por ultimo cuando para que el usuario sea creado satisfactoriamente, se crea **cost newUser**

el cual primero crea el **ID** propio usando **Date.now**, luego se colacan el email y contraseña que se recopilaron antes, (**Dato importante** se usa la variable que se creo antes que usaba bcrypt al costado de la variable password - contraseña), cuando este proceso se este realizando el servidor respondera de 2 maneras: Si todo esta bien, se mandara codigo 201 indicando que ya esta registrado, y si ocurre algún problema interno en el servidor se manda codigo 500, indicando la falla de esta.

Figura 2: Imagen de la prueba donde se hace el registro del usuario



- **2.** Endpoint POST /api/auth/login: Valida las credenciales comparando los hashes mediante bcrypt.compare(). Si es exitoso, crea la propiedad req.session.userId = user.id.
- DEMOSTRACION DEL CODIGO DONDE SE MUESTRA LA COMPARACION DE LAS CREDENCIALES:

```
// Endpoint POST /api/auth/login
// Ver si los datos son correctos o no
app.post('/api/auth/login', async (req, res) => {
    try {
        const { email, password } = req.body;

        if (!email || !password) {
            return res.status(400).json({ status: 'fail', message: 'Email y contrasena
                requeridos' });
        }

        const user = users.find(u => u.email === email);
        if (!user) {
            return res.status(401).json({ status: 'fail', message: 'Algo esta invalido'
                });
        }

        // Comparar contrasenas mediante bcrypt.compare()
        const isMatch = await bcrypt.compare(password, user.password);
        if (!isMatch) {
            return res.status(401).json({ status: 'fail', message: 'Algo esta invalido'
                });
        }
```

- **Explicación del codigo:** primero se declara el verbo **HTTP POST**, se colocan los parametros

que tendra, luego se crean 2 constantes que son **email** y **password**, con ello se usa en condicionador if para saber si se esta colocando el email y password en su respectivo lugar, si no es asi se devuelve codigo 400, indicando que falta llenar el campo, luego se crea **const user** esta variable se llena de datos usando el verbo anterior de registro, donde se compara el email ingresado con el que se registro, si no se coincide se devuelve **codigo 401** indicando que algo esta mal, luego se crea la variable: **isMatch** donde se usa la comparacion de los hashes usando **bcrypt compare** y se coloca la contraseña que genero la encriptacion verificandola con la contraseña que se esta colocando, si algo falla en la comparacion se devuelve **codigo 401** indicando que algo salio mal.

- DEMOSTRACIÓN DEL CODIGO DONDE PASA LA COMPARACION Y SE CREA LA PROPIEDAD **REQ SESION USERID**:

```
        // Crear req.session.userId si todo esta bien
        req.session.userId = user.id;

        return res.json({
            status: 'success',
            message: 'Inicio de sesion exitoso',
            user: { id: user.id, email: user.email }
        });
    } catch (error) {
        return res.status(500).json({ status: 'error', message: 'Error interno en el
            servidor' });
    }
});
```

- **Explicacion del codigo:** Tomar en cuenta que se comparo antes las contraseñas, por lo cual se va al siguiente paso donde se crea el usuario con **req.session.userId=user.id**, se devuelve un mensaje de confirmacion al servidor, y al usuario un mensaje de **inicio de sesion exitoso**, luego se define el **user** donde se integra el ID y EMAIL, por si ocurre algun problema con el servidor se lanza codigo 500, indicando que hubo un problema con el servidor.

4.3. Cierre de Sesión y Recurso Protegido:

- A continuación se mostrara la implementacion del **authMiddleware** de autorizacion, luego se implementan 2 Endpoint uno de verbo HTTP **GET** y **POST** el primero da los datos al usuario, y el segundo destruye la sesion del servidor y limpia la cookies.
- **1.** Middleware authMiddleware: Verifica si req.session.userId existe. Si no existe, interrumpe el flujo y retorna 401 Unauthorized.
- DEMOSTRACION DEL CODIGO EN NODE.JS

```
// Middleware de Autorizacion
function authMiddleware(req, res, next) {
    if (req.session && req.session.userId) {
        return next(); // Usuario autenticado
    }
    return res.status(401).json({
        status: 'fail',
        message: 'Unauthorized: acceso no autorizado'
    });
}
```

- **Explicacion del codigo:** primero se declara la funcion **authMiddleware** con los parametros **req**, **res** y **next**, luego se usa un condicionador if que verifica que exista **req.session** y que dentro de ella exista **req.session.userId**, esto solo pasa si el usuario ya inicio sesion. Si se cumple, se ejecuta **next()** que deja pasar la peticion al siguiente paso, que es el endpoint. Si no se cumple, no se llama a next(), se interrumpe el flujo y se devuelve **codigo 401** con un mensaje indicando que el acceso no esta autorizado.
- **2.** Endpoint Protegido GET /api/perfil: Protegido por authMiddleware. Retorna los datos del usuario autenticado.
- DEMOSTRACION DEL CODIGO EN NODE.JS

```
// Endpoint protegido GET /api/perfil
app.get('/api/perfil', authMiddleware, (req, res) => {
    const user = users.find(u => u.id === req.session.userId);

    if (!user) {
        return res.status(404).json({ status: 'fail', message: 'Usuario no encontrado'
            });
    }

    // Se devuelven los datos sin la contrasena
    return res.json({
        status: 'success',
        user: { id: user.id, email: user.email }
    });
});
```

- **Explicacion del codigo:** primero se declara el verbo **HTTP GET** con la ruta **/api/perfil**, y se coloca **authMiddleware** antes de la funcion principal, por lo que el endpoint solo se ejecuta si el usuario tiene sesion activa. Luego se crea **const user**, donde se busca en el arreglo de usuarios el que tenga el mismo id guardado en **req.session.userId**. Si no se encuentra, se devuelve **codigo 404** indicando que el usuario no existe. Si se encuentra, se devuelve un mensaje de **success** con el **id** y el **email** del usuario, sin incluir la contraseña para no exponer el hash.
- **3.** Endpoint POST /api/auth/logout: Destruye la sesión del servidor (req.session.destroy()) y limpia la cookie de sesión.
- DEMOSTRACION DEL CODIGO EN NODE.JS

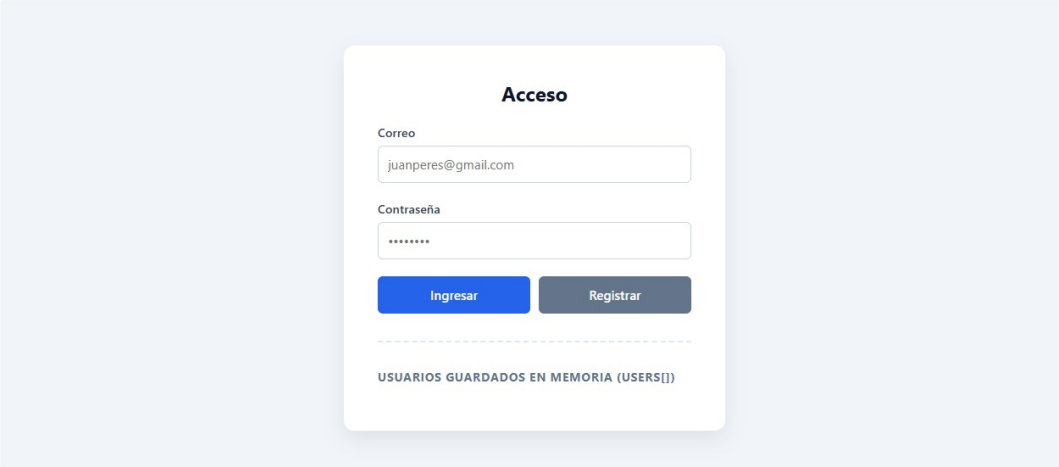
```
// Endpoint POST /api/auth/logout
app.post('/api/auth/logout', (req, res) => {
    req.session.destroy((err) => {
        if (err) {
            return res.status(500).json({ status: 'error', message: 'No se pudo cerrar
                la sesion' });
        }
        res.clearCookie('connect.sid'); // Limpia la cookie de sesion
        return res.json({ status: 'success', message: 'Sesion cerrada exitosamente' });
    });
});
```

- **Explicacion del codigo:** primero se declara el verbo **HTTP POST** con la ruta **/api/auth/logout**, luego se usa **req.session.destroy()** que elimina la sesion guardada en el servidor. Si ocurre un error al destruirla, se devuelve **codigo 500** indicando que no se pudo cerrar la sesion. Si todo

sale bien, se usa **res.clearCookie('connect.sid')** para limpiar la cookie de sesion del navegador, se esta usando **connect.sid** ya que ese es el nombre por defecto que usa express-session, y se devuelve un mensaje de **sesion cerrada exitosamente**.

4.4. Resultado final de la pagina web.

Figura 3: Visualizacion final de la pagina.



4.5. Estructura de laboratorio 06

- El contenido que se entrega en este laboratorio es el siguiente:

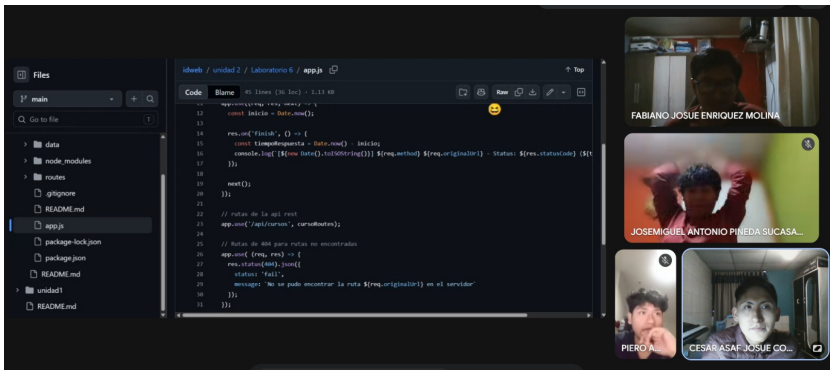
```
.
|----backend
     |----package.json
     |----priv
     |----pub
          |----css
          |----fonts
          |----img
          |----js
          +----others
     +----README.md
|----bd
     |----bd.sql
     +----README.md
|----frontend(opcional)
     |----package.json
     |----priv
     |----pub
          |----css
          |----fonts
          |----img
          |----js
          +----others
     +----README.md
|----img
```

```
     +----enrollmentsv1.0.png
|----informes
     |----IDWeb_lab07_avance_proyecto.pdf
+----README.md
```

4.6. ANEXO

- Se realizo reuniones en google meet para comprobar cada parte del codigo, y repartirse partes para corregir o agregar algo.

Figura 4: Reunion virtual para coordinacion.



5. Cuestionario:

5.1. ¿Por qué nunca deben almacenarse contraseñas en texto plano ni utilizar funciones de cifrado bidireccional (AES/RSA) en lugar de algoritmos de Hashing (bcrypt/Argon2)?

- Si todas las contraseñas se almacenan en texto plano puede ocurrir que la base de datos se filtre y todas las contraseñas quedan expuestas de inmediato.
- El cifrado bidireccional es reversible: En cuestion si alguien obtiene la clave puede recuperar las contraseñas originales.
- El hashing es unidireccional y no se puede revertir. Además, el salado hace que dos contraseñas iguales generen hashes distintos, y su lentitud dificulta los ataques de fuerza bruta.

5.2. ¿Qué función cumplen los atributos de seguridad de una cookie: httpOnly, secure y sameSite en la prevención de ataques XSS y CSRF?

- **httpOnly**: impide que JavaScript lea la cookie, por lo que un ataque XSS no puede robar la sesión.
- **secure**: la cookie solo se envía por HTTPS, evitando que sea interceptada en la red.
- **sameSite**: restringe el envío de la cookie en peticiones que vienen de otros sitios, previniendo ataques CSRF.

5.3. Explique la diferencia conceptual de escalabilidad entre la autenticación basada en Sesiones en Servidor (Stateful) y la autenticación basada en Tokens JSON Web Tokens - JWT (Stateless).

- **Stateful (sesiones):** el servidor guarda el estado de cada sesión. Al escalar a varios servidores, estos deben compartir las sesiones en cuestion todos estan en un almacen en comun por lo cual añade complejidad.
- **Stateless (JWT):** el token firmado contiene la información del usuario y el servidor solo verifica la firma, sin guardar nada. Escala con facilidad porque cualquier servidor puede validar la petición.

6. COMMITS:

- El contenido de los commits se mostrara a continuacion:

```
e47aba7 Corrigiendo error en el readme
a3625b6 Agregando al readme la autocalificacion
cd8b7f6 Actualizando el .gitignore
4f155da Corrigendo la data que se mostrara
f9c2744 Creando la carpeta del informe
6270d07 Lab 06: Se colocaron cursos del semestre del estudiante por defecto
2c7faa7 Lab 06: Permitimos "unir" el index con el PORT
83fffb8 Lab 06: Subimos la imagen de fondo
3acce31 Lab 06: Frontend - Colocamos la Imagen de Fondo y acabamos con pequeos detalles del
    formulario y el listado
6cd210b Lab 06: Agregamos una imagen de fondo, cambiamos el color de los botones de guardado
    y busqueda, terminamos de configurar el Frontend
958f0f4 Lab 06: Frontend - Creamos un filtro, listado y busqueda por ID
10807b8 Lab 06: Frontend - Creamos el formulario
52e35b4 Lab 06: Empezamos a trabajar el Frontend de la pgina, styles.css
848ce84 Lab 06: Empezamos a trabajar el Frontend de la pgina, html
40b27ee Lab 06: arreglamos el controller "cursoData"
8b10694 Lab 06: Arreglamos el nombre de "cursosData" a "cursoData" ya que en controller no
    lo reconoce por esa "s"
3bc18d9 Eliminado archivo que esta mal
57ae054 Lo que aparece en la terminal
0659ad6 Prueba de los datos incompletos
86ab4fa Evidencia de la consulta de un curso inexistente
07a34cc Editando el folder
9d06086 Prueba de la eliminacion
df32c52 Prueba de la actualizacion
a8f7709 Creando curso
2fa7121 Prueba de las consultas del curso
10e5487 Prueba de la filtracion por creditos
63fac84 eliminando el archivo errado
79b06aa Evidencia de la lista de cursos
d8cdf37 Creando la carpeta pruebas
12b78a4 Corrigiendo error
2795d04 Corrigiendo error en rutas de 404 para que el servidor funcione sin problemas
ad6fb23 Aadiendo node_modules a gitignore
aec203a Aadiendo middleware y manejo de rutas a app.js
8213c25 Aadiendo los cursos a cursosData.js
4092ea8 Aadiendo lo necesario a cursoRoutes.js
```

```
546f854 Aadiendo gitignore y aadiendo lo necesario a cursoController.js
fe769ac creando controllers, data, routes y app.js
af67b51 Creando package.json e instalando express
a2338e2 cambiando nombre de laboratorio 5 a lab 6
```

7. Rúbricas

7.1. TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

Tabla 1: Tipo de Evaluacion

Evaluacion	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.
Tecnicas:	Auditoría de seguridad de código, pruebas del flujo Registro-Login-Privado-Logout.
Instrumentos:	Rúbrica analítica de evaluación de laboratorio y Colección Postman.

7.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

	Nivel			
Puntos	Deficiente 25 %	En Proceso 50 %	Bueno 75 %	Excelente 100 %
7.0	1.75	3.5	5.25	7.0
3.0	0.75	1.5	2.25	3.0

Tabla 3: Tabla de evaluacion de participacion de cada integrante

Grupo F3	**Alumnos**
Integrante	Piero Alejandro Arosquipa Laury
Integrante	Fabiano Josue Enriquez Molina
Integrante	Josemiguel Pineda Sucasaca
Integrante	César Asaf Josue Coricaza López

Tabla 4: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. Módulo de Autenticación y Hash (35 %)	Implementa la autenticación completa. Cifra contraseñas obligatoriamente con bcryptjs y compara hashes de forma asíncrona impecablemente.	7	X	6	
2. Manejo de Sesiones y Protección (35 %)	Configura express-session con cookies httpOnly. Protege adecuadamente las rutas privadas mediante middlewares de autorización.	7	X	5	
3. Seguridad y Variables de Entorno (15 %)	Aisla completamente secretos en ‘.env‘. No expone datos sensibles ni contraseñas en las respuestas JSON devueltas.	3	X	2	
4. Cuestionario Teórico (15 %)	Demuestra conocimiento amplio en vectores de ataque OWASP, hashing, salado y banderas de protección de cookies.	3	X	2	
Total		20.0		15	

8. Referencias

- https://masteringjs.io/tutorials/express/req
- https://expressjs.com/es/guide/using-middleware.html
- https://github.com/github/gitignore
- https://developer.mozilla.org/en-US/docs/Web/HTTP/Status